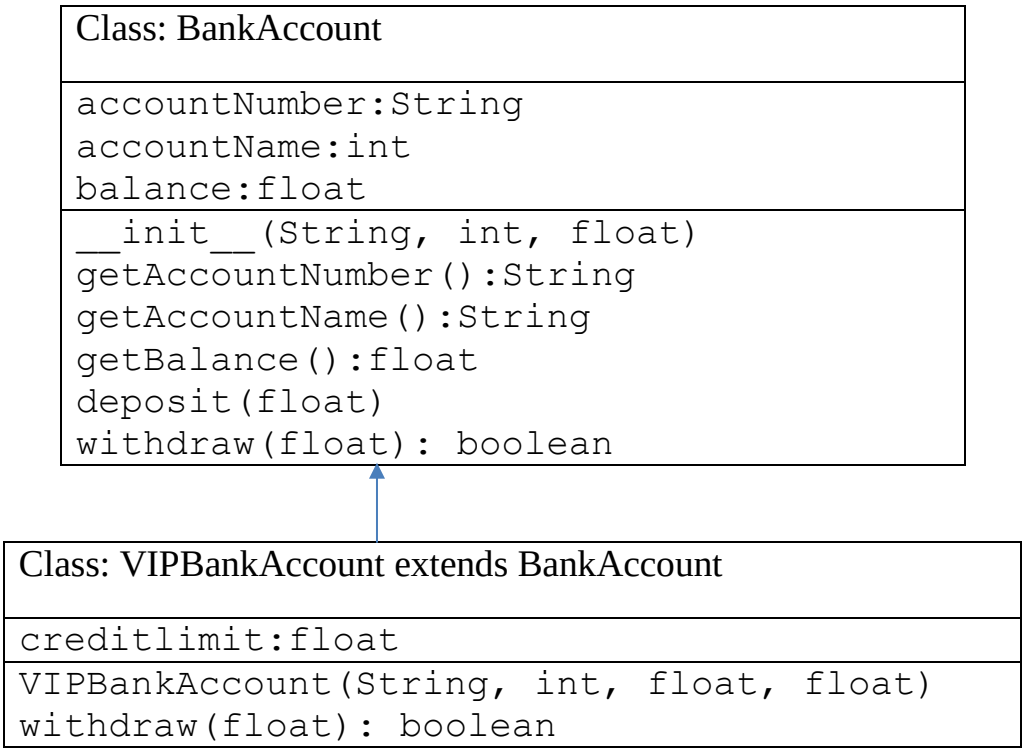
Software Development – Inheritance

Python programming exercises

1. Write the class Marketer to accompany the other law firm classes described in lecture. Marketers make $50,000 ($10,000 more than general employees) and have an additional method called advertise that prints "Act now, while supplies last!" Make sure to interact with the superclass as appropriate.

2. Write a class HarvardLawyer to accompany the other law firm classes described in the lecture. Harvard lawyers are like normal lawyers, but they make 20% more money than a normal lawyer, they get 3 days more vacation, and they have to fill out four of the lawyer's forms to go on vacation. That is, the getVacationForm method should return "pinkpinkpinkpink". Make sure to interact with the superclass as appropriate.

3. Create a `VIPBankAccount` class based on the above UML notation.



4. See the mini project released this week. Make sure that the systems works with VIPBankAccount you developed in Task 3.

5. Create consider the task of representing types of tickets to campus events. Each ticket has a unique number and a price. There are three types of tickets: walk-up tickets, advance tickets, and student advance tickets. Figure.1 illustrates the types:

    - Walk-up tickets are purchased the day of the event and cost $50.
    - Advance tickets purchased 10 or more days before the event cost $30, and advance tickets purchased fewer than 10 days before the event cost $40.
    - Student advance tickets are sold at half the price of normal advance tickets: When they are purchased 10 or more days early they cost $15, and when they are purchased fewer than 10 days early they cost $20.

    Tasks:

    a) Implement a class called Ticket that will serve as the superclass for all three types of tickets. Define all common operations in this class, and specify all differing operations in such a way that every subclass must implement them. No actual objects of type Ticket will be created: Each actual ticket will be an object of a subclass type. Define the following operations:

        - The ability to construct a ticket by number.
        - The ability to ask for a ticket's price.
        - The ability to println a ticket object as a String. An example String would be "Number: 17, Price: 50.0".

    b) Implement a class called WalkupTicket to represent a walk-up event ticket. Walk-up tickets are also constructed by number, and they have a price of $50.

    c) Implement a class called AdvanceTicket to represent tickets purchased in advance. An advance ticket is constructed with a ticket number and with the number of days in advance that the ticket was purchased. Advance tickets purchased 10 or more days

before the event cost $30, and advance tickets purchased fewer than 10 days before the event cost $40.

d) Implement a class called StudentAdvanceTicket to represent tickets purchased in advance by students. A student advance ticket is constructed with a ticket number and with the number of days in advance that the ticket was purchased. Student advance tickets purchased 10 or more days before the event cost $15, and student advance tickets purchased fewer than 10 days before the event cost $20 (half of a normal advance ticket). When a student advance ticket is printed, the String should mention that the student must show his or her student ID (for example, "Number: 17, Price: 15.0 (ID required)").

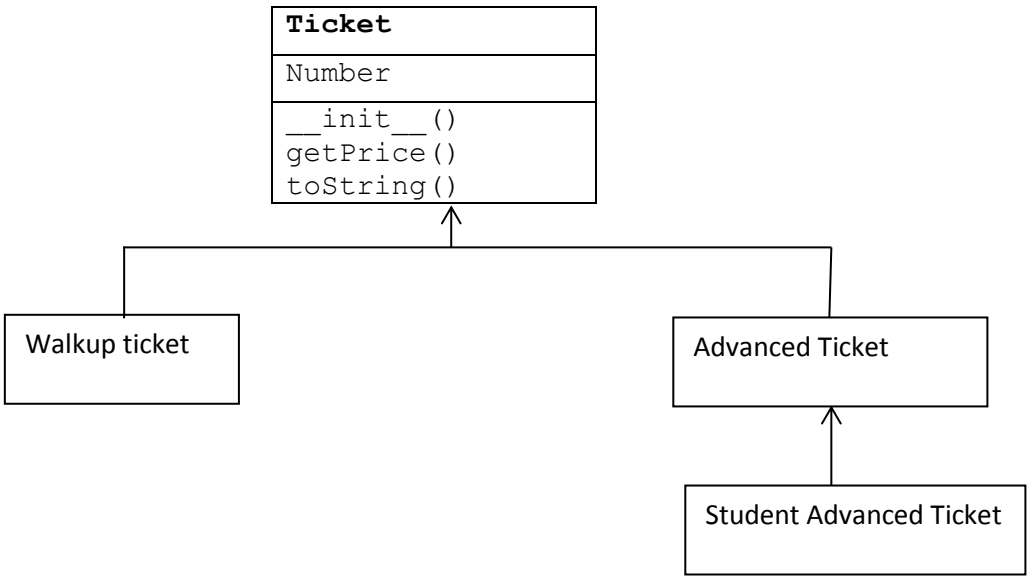


Figure 1.